

FROM SUMMATIVE MCQ EXAMS TO FORMATIVE PRACTICE: MC-TEST FOR AI-ASSISTED ITEM GENERATION, LEARNER ANALYTICS, AND ADAPTIVE PACING

R. Bimberg¹, P. Kubica², K. Quibeldey-Cirke²

¹*Technische Hochschule Mittelhessen (GERMANY)*

²*IU International University (GERMANY)*

Abstract

Multiple-choice (MC) exams scale to large cohorts, but many platforms provide little more than scores and item-level correctness. We present MC-Test, a privacy-aware web platform for formative MC practice that integrates large language model (LLM)-assisted item generation, structured feedback, learner-facing analytics, and adaptive pacing. MC-Test uses a didactic item model that links each item to a learning objective, topic, weight, and an operationalized cognitive level (restricted Bloom-inspired cognitive-level labels: reproduction, application, analysis). Items are generated through a finite-state prompt workflow and emitted as schema-conformant JSON; automated validation, repair, and option harmonization support robust import, instructor review, and downstream item statistics. After each session, learners receive rationales for every option, a mini-glossary, and exportable learning objectives and study resources.

Four dashboard views—Topic Performance Chart, Cognitive Radar Chart, Concept Mastery Columns, and Topic × Cognitive Heatmap—connect results to topics and cognitive levels. Configurable cooldowns are intended to reduce rapid guessing; a Time-Critical Override relaxes pacing when remaining time per item becomes critical. In the described on-premises deployment, prompts and learner data are processed within institutional infrastructure, and pseudonymous access supports participation without personal accounts. A pilot usability study (SUS, N = 20) provides preliminary evidence of acceptable perceived usability (mean 70.38/100, rated “OK” on the adjective rating scale).

MC-Test is open-source software (MIT): <https://github.com/kqc-real/streamlit>.

Keywords: Artificial Intelligence in Education, Multiple-Choice Question Generation, Bloom’s Taxonomy, Learner-Facing Learning Analytics, Self-Regulated Learning, Rapid Guessing Detection, Adaptive Pacing, Privacy-Aware Local Language Models, Formative Assessment.

1 INTRODUCTION

Multiple-choice (MC) questions are widely used in university assessment because they scale well, support objective scoring, and are easy to administer in large cohorts. However, many commonly used MC practice systems provide little more than a total score and item-level correctness, offering limited support for metacognitive reflection and strategic exam preparation. Learners often receive feedback that is too coarse to inform their study strategies: they know that they failed, but not why.

Large language models (LLMs) can now generate MC items at scale. This shifts the bottleneck from authoring to quality assurance: items must be valid, appropriately difficult, and aligned with learning objectives. If metadata such as topic, difficulty, and cognitive level are recorded explicitly, MC practice can also yield learner-facing analytics instead of only scores. Prior work shows that LLM-generated MCQs can achieve competitive psychometric properties in some settings, but it also underlines the need for systematic, item-level validation rather than assuming quality from generation alone [1]. Scaria et al. examine LLM question generation across Bloom levels and emphasize both potential and evaluation limitations [2].

We built MC-Test, a web-based platform that combines:

- A didactic item model linking each item to learning objectives, topic, and cognitive-level labels (Bloom levels 1–3).
- An LLM pipeline that generates schema-conformant JSON via a finite-state interaction and automated validation/repair.

- Learner-facing analytics that visualize performance by topic and cognitive level.
- Pedagogical controls (cooldowns, reflection windows, adaptive pacing) to mitigate rapid guessing.

MC-Test initially relied on commercial LLM APIs and was later migrated to a local LLM backend via Ollama.

We address three research questions:

- RQ1: How can cognitive-level classifications be operationalized in a learner-facing assessment platform?
- RQ2: How can LLMs be embedded via robust prompt engineering to support reliable item generation and future psychometric quality checks?
- RQ3: Which design considerations arise when migrating to a local LLM backend?

RQ1 is addressed through the cognitive-level taxonomy and dashboards; RQ2 through the schema-constrained prompt workflow; and RQ3 through the local-backend migration analysis. Accordingly, this paper contributes three elements: (1) an operational didactic item model that connects learning objectives, cognitive-level labels, and formative rationales; (2) a schema-constrained LLM generation pipeline that produces parseable JSON and supports automated validation, repair, and future item analysis; and (3) deployment lessons from an on-premises Ollama-based architecture for privacy-sensitive teaching contexts. We conducted a pilot usability study and outline planned log-based evaluation of rapid guessing, label reliability, and item quality.

MC-Test targets formative MC practice in STEM (Bloom levels 1–3), excluding higher-order tasks because prior work reports reliability concerns for higher-order MCQ generation and validation and emphasizes the need for strict quality assurance [1], [3]. Preliminary usability evidence is available (SUS = 70.38, “OK”) but learning effects and behavioral changes require follow-up studies (Section 6). Cognitive labels will be double-coded (kappa planned), and human item review remains mandatory. Pacing is course-configurable for accessibility.

2 THEORETICAL BACKGROUND AND RELATED WORK

Cognitive Taxonomies and AI Validity: Bloom’s taxonomy has long provided a framework for differentiating cognitive complexity. While LLMs often perform well on lower taxonomic levels (remember, understand), recent studies report reliability challenges when models generate or validate higher-order evaluation or creation tasks without human-in-the-loop verification [1]. Consequently, automated MC generation requires strict constraints and human review to support semantic validity [3].

Learner-Facing Learning Analytics & Self-Regulation: Learner-facing (also termed student-facing) learning analytics dashboards aim to provide learners with interpretable visualizations to support self-regulated learning [4]. However, research suggests that pure data visualization is often insufficient; effective systems must provide active guidance or “nudges” to translate insights into behavioral change [5]. In addition, prior work provides design and evaluation recommendations for learner-facing learning analytics dashboards [6]. MC-Test responds to this design challenge by combining analytics with pacing and reflection mechanisms.

Test-Taking Effort and Rapid Guessing: In computer-based assessment, unusually short response times can indicate reduced test-taking effort (e.g., rapid guessing) rather than lack of knowledge. Response-time effort metrics and effort-moderated models use response times to flag or down-weight such responses and provide practical guidance for setting response-time thresholds and interpreting rapid-guessing indicators in low-stakes contexts [7], [8].

Privacy in Generative AI: The use of commercial LLM APIs in education raises significant privacy concerns regarding data leakage and GDPR compliance [9]. Recent work highlights the “privacy paradox” of LLMs, suggesting that on-premises deployment may mitigate selected data-transfer and governance risks in sensitive educational contexts [10].

3 METHODOLOGY AND SYSTEM DESIGN: THE MC-TEST PLATFORM

3.1 Pedagogical Requirements

MC-Test was designed for formative assessment in STEM courses. Key requirements included:

- Cognitive transparency: Items must be classified by cognitive level.
- Actionable feedback: Explanations must link to learning objectives.
- Exam readiness: Support for time management skills.
- Didactic control: Educators must retain control over item quality.

3.2 System Architecture

The application is implemented with the Python framework Streamlit and has been migrated from a cloud-based prototype to a containerized microservice architecture. The stack is orchestrated with Docker Compose and deployed on an on-premises institutional server to support institutional data sovereignty. Key components include:

- Data Persistence: A PostgreSQL database replaces lightweight file-based storage to handle concurrent user sessions and complex relational data (user profiles, audit logs) reliably.
- Privacy-Compliant Rendering: To eliminate data leakage to external content delivery networks (CDNs), mathematical formulas are rendered using a self-hosted local MathJax instance.
- Local Inference Engine: A central architectural shift is the integration of Ollama as a local LLM backend. This allows the platform to execute open-weight models (e.g., a DeepSeek-family model) within institutional infrastructure. In the described deployment configuration, prompts and learner data are processed within institutional infrastructure. The migration shows that local inference requires not only infrastructure changes but also stricter schema validation, model-specific prompt calibration, and operational governance for logs, access, and retention.

3.3 Cognitive-Level Taxonomy for AI-Generated MCQs

MC-Test uses a restricted three-level taxonomy inspired by Bloom: (1) Reproduction (weight 1): recall of facts, definitions, or simple algorithms; (2) Application (weight 2): apply known concepts in slightly varied contexts; and (3) Analysis (weight 3): interpret data, compare, or infer. Levels 4–6 (Evaluation, Creation) are excluded. Prior work reports reliability concerns for higher-order tasks in closed MC formats [1] and indicates that strict constraints are needed for semantically valid MC generation [3]. Limiting items to levels 1–3 improves verifiability and reduces the risk of semantic hallucinations, but human review and item analysis remain necessary for factual and psychometric quality.

3.4 Operationalization and Label Reliability

Learner-facing analytics rely on stable cognitive-level labels. MC-Test therefore operationalizes Bloom levels 1–3 with a short rubric and a verification step. In our workflow: (1) instructors specify the learning objective and target level; (2) the LLM generates an item and rationales aligned with that objective; (3) instructors review the item before release. To quantify label reliability, two instructors will double-code a sample using the same rubric. We will report percent agreement and Cohen's kappa and treat low agreement as a warning signal before interpreting cognitive profiles at scale.

3.5 Prompt Architecture

Item generation is controlled by a context-engineered system instruction set that frames the model as an “Interactive MCQ Generator.” Here, context engineering refers to the deliberate design of (i) an interaction protocol that elicits all generation parameters and (ii) hard output constraints that make generated items machine-checkable.

MC-Test uses a finite-state workflow with three phases. (1) Configuration: the model collects parameters stepwise (topic, target audience, item count, difficulty/weight profile, option format, optional curricular context) and asks for explicit confirmation. (2) Internal blueprinting: the model performs an internal consistency check (e.g., difficulty weights match the requested profile; option/answer constraints are satisfiable) without exposing intermediate reasoning; this step checks that cognitive level, option constraints, and answer keys are mutually consistent before output is accepted. (3) Schema-driven output: the model returns a single, strictly parseable JSON object that follows explicit escaping and formatting rules, including stable item IDs and a schema version for forward compatibility.

For robustness across cloud and local backends, the JSON is subjected to automated schema and semantic validation (e.g., required fields, unique options, valid answer indices), with an optional repair loop that prompts the model to apply minimal edits when violations are detected. The repair loop targets structural errors and semantic drift rather than treating generated text as valid by default.

Option harmonization: During experimental use, we observed a recurring item-writing flaw in LLM-generated MCQs: the keyed option was sometimes more detailed, longer, or syntactically more salient than the distractors, thereby creating unintended cues for test-wise learners. This observation aligns with recent docimological analyses of LLM-generated MCQs, which identify “key too long” and grammar-based cues as quality flaws that can make the correct answer easier to detect without the intended knowledge [11]. MC-Test therefore applies a second revision prompt to the generated JSON output that preserves the answer key and semantic content while harmonizing sentence length, syntactic structure, and specificity across all answer options before instructor review.

Post-production learning objectives: After generating a question set, the LLM proposes a small set of topic labels (typically ~10) to structure the item bank. An additional prompt derives Bloom-aligned learning objectives both for the overall question set and for each individual item, using the assigned topic labels and cognitive levels. We compile these objectives into a learner-facing study resource that summarizes learning targets across topics and levels.

4 LEARNER-FACING ANALYTICS AND PEDAGOGICAL CONTROL

After a test, MC-Test shows a learner-facing analytics dashboard to support metacognitive reflection. Instead of a single score, it provides four views that cover curricular topics, cognitive levels, and concept mastery. By translating score patterns into topic-, level-, and concept-specific cues, the dashboard is designed to support self-assessment and subsequent study planning rather than merely reporting outcomes. In this section, topics are coarse curriculum units; concepts are fine-grained learning targets within a topic.

The four views form a diagnostic funnel from overview to detail. The heatmap may help identify avoidance patterns, such as systematically unanswered topic-level combinations. Topic bars and the Cognitive Radar Chart separate content gaps from process gaps, while Concept Mastery Columns translate patterns into concrete remediation targets. The intended design goal is to move learners from global self-judgments to actionable diagnoses [5], [6].

4.1 Topic Performance Chart

Fig. 1 shows the Topic Performance Chart, which aggregates results by topic area (e.g., regression, classification, evaluation). Each stacked bar shows correct, incorrect, and unanswered items; the x-axis labels report answered/total to indicate evidence volume. This view highlights topics that need remediation and topics that may have been skipped.

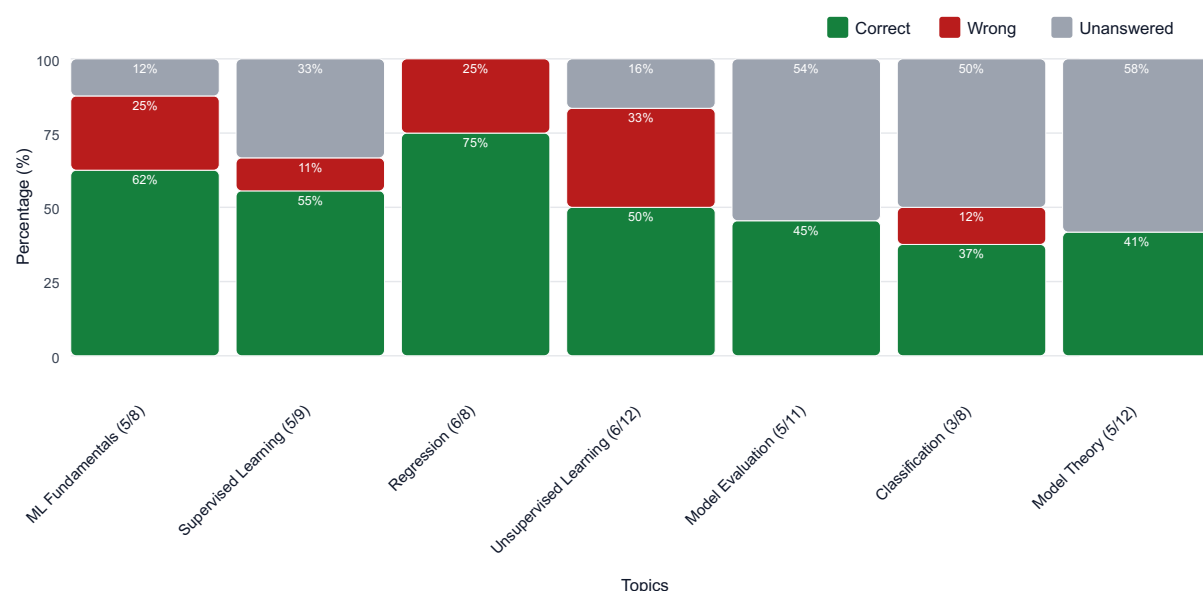


Figure 1. Topic Performance Chart: correct, incorrect, and unanswered responses by topic.

4.2 Cognitive Radar Chart

Fig. 2 shows the Cognitive Radar Chart, which summarizes performance across the three levels (reproduction, application, analysis). It indicates whether a learner mainly recalls facts or can transfer and reason with knowledge. Balanced shapes suggest even development; strong asymmetries highlight targeted development zones. For reflection, these patterns may be interpreted as recall-dominant, application-dominant, or more balanced cognitive profiles.



Figure 2. Cognitive Radar Chart: performance across Bloom levels 1–3.

4.3 Concept Mastery Columns

Fig. 3 shows the Concept Mastery Columns. MC-Test groups items by tagged concepts (e.g., cross-validation, regularization) and classifies each concept as understood, not understood, or not attempted. A concept counts as understood when at least 70% of its items are correct; otherwise, it is flagged to surface misconceptions without overinterpreting single items. The threshold is configurable and used as a heuristic rather than a validated mastery cut score. This micro-level view is intended to turn test outcomes into concrete study targets.

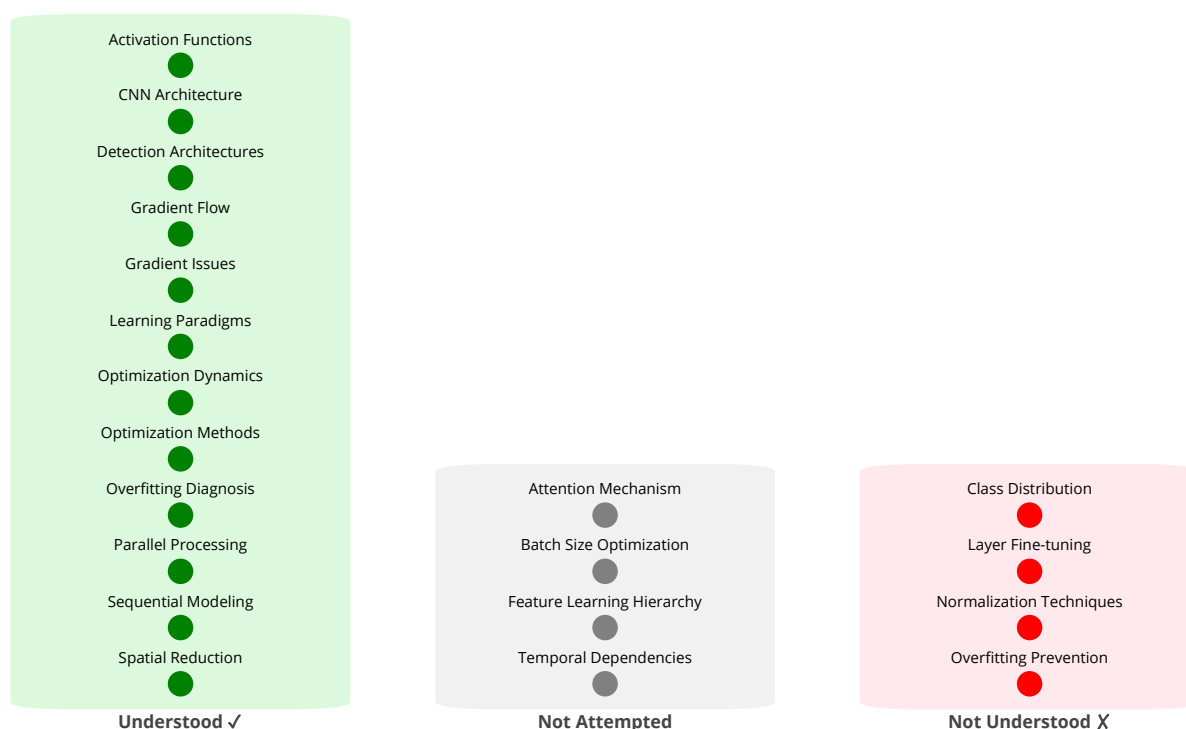


Figure 3. Concept Mastery Columns: concepts grouped using a 70% correctness threshold.

4.4 Topic × Cognitive Heatmap

Fig. 4 shows the Topic × Cognitive Heatmap, which combines topics (rows) and cognitive levels (columns). Cells encode performance and evidence volume: unanswered items count as 0, and each cell reports answered/total. This is intended to discourage strategic skipping and to help learners locate

whether difficulties are topic-specific or level-specific. By crossing topic and level, the heatmap makes it easier to distinguish a content gap in one curriculum area from a broader process-level difficulty.

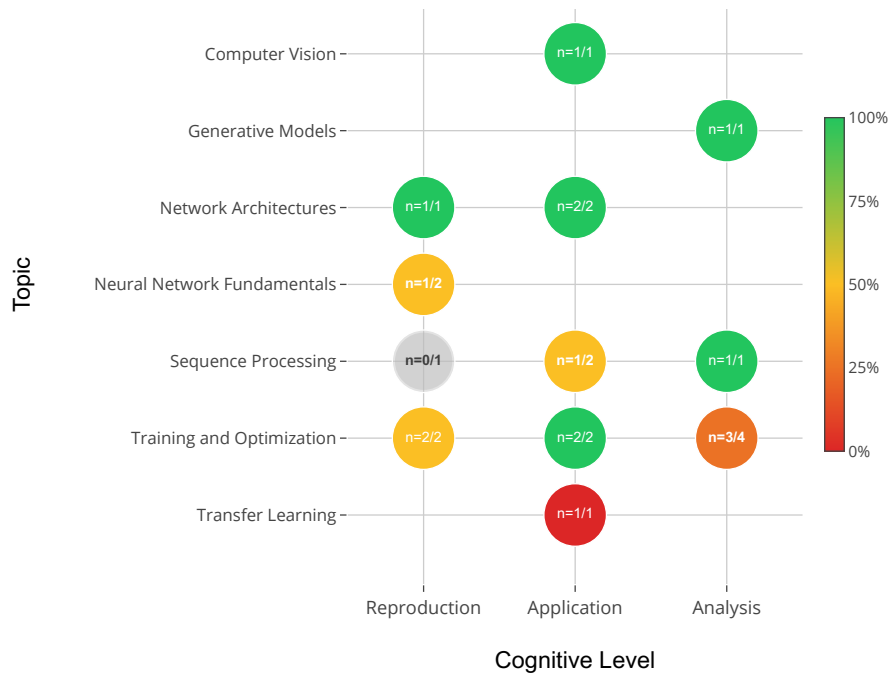


Figure 4. Topic $\times$ Cognitive Heatmap: answered/total evidence and score by topic and cognitive level.

Together, the four views triangulate topic coverage, cognitive level, and fine-grained concept mastery, helping learners move from global scores to actionable study targets.

4.5 Feedback and Explanations

For complex items, explanations are extended with step-by-step reasoning. To support vocabulary acquisition, a two-tiered glossary concept is used: a mini-glossary field for immediate help and a cumulative post-test PDF glossary.

4.6 Pedagogical Pacing Mechanisms: Cooldowns and Time-Critical Override

MC-Test manages pacing to encourage reading and reflection and to deter impulsive responding (e.g., rapid guessing). We treat these pacing nudges as design hypotheses and evaluate their behavioral effects empirically, rather than claiming to enforce deep processing.

The pacing control layer provides three mechanisms. Pre-Answer Cooldown (reading support): the Submit button stays disabled for a short, item-specific time after loading; the duration depends on the item weight and text length. Post-Answer Cooldown (reflection support): after submission, navigation to the next item is delayed to create a review window for formative feedback [12]; the duration scales with explanation length (+10 s standard, +20 s extended). Adaptive pacing (Time-Critical Override): because pacing can hinder learners under time pressure, the system can disable cooldowns when time becomes critical.

The system monitors remaining time per remaining item. If it falls below a threshold (e.g., less than 15 seconds per item), the Time-Critical Override disables cooldowns so learners can finish the attempt. Because fixed pacing can disadvantage some learners (e.g., assistive-technology users or students with accommodations), all cooldowns are configurable per course and can be disabled or adjusted as needed. In short, the Time-Critical Override prioritizes completion and is intended to reduce pacing-related disadvantage when time pressure is high.

5 IMPLEMENTATION AND MIGRATION TO LOCAL LLM BACKEND

MC-Test was migrated from commercial cloud APIs to an on-premises institutional deployment with Ollama as local LLM backend. This shift reduces data-transfer risks and supports institutional data-sovereignty goals, improves deployment control, and reduces dependence on API pricing, rate limits,

and external availability. The system prompt was refactored for local model behavior, prioritizing JSON validity, explicit schema constraints, and repair robustness. Data minimization is applied by storing assessment responses, timestamps, aggregate scores, minimal operational metadata, and security logs only; prompts avoid personal identifiers, access is restricted to authorized staff, and retention policies limit storage duration. Pseudonymous participation supports progress tracking without personal accounts.

The open-source Streamlit application is released under the MIT License.

6 PRELIMINARY EVALUATION AND OUTLOOK

6.1 Planned Evaluation Design

Planned STEM deployments will combine system logs, pre/post questionnaires, and interviews. Comparative or phased designs will test whether pacing affects rapid-guessing indicators, feedback viewing, study strategies, and learning-related outcomes. Cognitive labels will be double-coded with a rubric and reported using percent agreement and Cohen's kappa; basic item statistics will flag ambiguous or low-quality items before cognitive profiles are interpreted at scale.

6.2 Preliminary Results: System Usability

To provide preliminary evidence on perceived usability, we conducted a standardized usability evaluation using the System Usability Scale (SUS) [13]. The survey was administered to a convenience pilot cohort of 20 participants in a prototype evaluation, yielding a mean SUS score of 70.38. Because the participant pool was not sampled for population-level inference, the result should be interpreted as preliminary usability evidence.

Using the adjective rating scale by Bangor et al. [14], the mean SUS score falls in the “OK” category, indicating acceptable perceived usability and a promising initial user experience. Importantly, SUS primarily captures perceived usability; it does not establish learning benefits or behavioral effects. These aspects will be examined in subsequent studies using log-based rapid-guessing indicators, learner feedback on fairness and autonomy, and (where feasible) pre/post-test learning outcome measures.

6.3 Limitations

The study contributes system design and preliminary usability evidence. Claims about reduced rapid guessing, improved metacognition, and learning outcomes require follow-up validation. Cognitive-level labels remain interpretive and require double-coding and item statistics. Pacing may affect autonomy and accessibility; therefore, course-level configuration and accommodation pathways are necessary.

7 CONCLUSION

MC-Test demonstrates how LLM-supported MC item generation can be embedded in a formative practice platform that meaningfully extends beyond scores: by combining schema-constrained generation, learner-facing analytics across topic and cognitive dimensions, and configurable pacing mechanisms, the system addresses well-documented gaps in large-cohort assessment practice.

The contribution at this stage is primarily design-oriented. The operational didactic item model makes cognitive classifications visible and actionable in learner-facing dashboards (RQ1); the finite-state prompt workflow with automated validation and repair supports robust, parseable item generation and prepares the ground for future psychometric analysis (RQ2); and the on-premises Ollama-based architecture provides a replicable blueprint for privacy-sensitive institutional deployments (RQ3). A pilot usability study (SUS = 70.38, N = 20) offers preliminary evidence that the platform is perceived as acceptably usable: <https://mc-test.streamlit.app/>.

What remains to be established is whether these design choices translate into the intended behavioral changes. Planned studies will examine rapid-guessing indicators from system logs, cognitive-label reliability via double-coding (Cohen's kappa), and—where feasible—pre/post learning outcomes in live STEM courses. The open-source release under the MIT License invites replication and adaptation across disciplinary contexts.

REFERENCES

- [1] S. Bhandari, Y. Liu, Y. Kwak, and Z. A. Pardos, "Evaluating the psychometric properties of ChatGPT-generated questions," *Computers and Education: Artificial Intelligence*, vol. 7, Art. 100284, 2024, doi: 10.1016/j.caeai.2024.100284.
- [2] N. Scaria, S. D. Chenna, and D. Subramani, "Automated Educational Question Generation at Different Bloom's Skill Levels Using Large Language Models: Strategies and Evaluation," in *Artificial Intelligence in Education (AIED 2024)*, LNAI vol. 14830, pp. 165-179, 2024, doi: 10.1007/978-3-031-64299-9_12.
- [3] M. Ebner, B. Brünner, N. Forjan, and S. Schön, "Ensuring Quality in AI-Generated Multiple-Choice Questions for Higher Education with the QUEST Framework," in *New Media Pedagogy: Research Trends, Methodological Challenges, and Successful Implementations - 3rd International Conference, NMP 2024, Revised Selected Papers*, L. Tomczyk, Ed., *Communications in Computer and Information Science*, vol. 2537. Cham: Springer, 2025, pp. 293–303, doi: 10.1007/978-3-031-95627-0_20.
- [4] R. Bodily and K. Verbert, "Trends and issues in student-facing learning analytics reporting systems research," in *Proc. 7th Int. Conf. Learning Analytics & Knowledge (LAK '17)*, Vancouver, BC, Canada, Mar. 2017, pp. 309-318, doi: 10.1145/3027385.3027403.
- [5] L. de Vreugd, A. van Leeuwen, R. Jansen, and M. van der Schaaf, "Learning Analytics Dashboard Design and Evaluation to Support Student Self-Regulation of Study Behaviour," *Journal of Learning Analytics*, vol. 11, no. 3, pp. 249–262, 2024, doi: 10.18608/jla.2024.8529.
- [6] I. Jivet, M. Scheffel, M. Specht, and H. Drachsler, "License to evaluate: Preparing learning analytics dashboards for educational practice," in *Proc. 8th Int. Conf. Learning Analytics & Knowledge (LAK '18)*, Sydney, NSW, Australia, Mar. 2018, pp. 31-40, doi: 10.1145/3170358.3170421.
- [7] S. L. Wise and C. E. DeMars, "An application of item response time: The effort-moderated model," *Journal of Educational Measurement*, vol. 43, pp. 19-38, 2006.
- [8] S. L. Wise, "Rapid-guessing behavior: Its identification, interpretation, and implications," *Educational Measurement: Issues and Practice*, vol. 36, no. 4, pp. 52–61, 2017, doi: 10.1111/emip.12165.
- [9] H. Drachsler and W. Greller, "Privacy and Analytics: It's a DELICATE Issue—A Checklist for Trusted Learning Analytics," in *Proc. of the 6th Int. Conf. on Learning Analytics & Knowledge*, Edinburgh, UK, 2016, pp. 89–98.
- [10] Y. Shanmugarasa, M. Ding, M. A. P. Chamikara, and T. Rakotoarivelo, "SoK: The Privacy Paradox of Large Language Models: Advancements, Privacy Risks, and Mitigation," in *Proc. of the 20th ACM Asia Conf. on Computer and Communications Security (ASIA CCS '25)*, New York, NY, USA: Association for Computing Machinery, 2025, pp. 425–441, doi: 10.1145/3708821.3733888.
- [11] C. Grévisse, M. A. S. Pavlou, and J. G. Schneider, "Docimological Quality Analysis of LLM-Generated Multiple-Choice Questions in Computer Science and Medicine," *SN Computer Science*, vol. 5, Art. 636, 2024, doi: 10.1007/s42979-024-02963-6.
- [12] N. E. Winstone and D. Carless, *Designing Effective Feedback Processes in Higher Education: A Learning-Focused Approach*. London: Routledge, 2019.
- [13] J. Brooke, "SUS: A 'Quick and Dirty' Usability Scale," in *Usability Evaluation in Industry*, P. W. Jordan, B. Thomas, B. A. Weerdmeester, and I. L. McClelland, Eds. London, U.K.: Taylor & Francis, 1996, pp. 189-194.
- [14] A. Bangor, P. T. Kortum, and J. T. Miller, "Determining what individual SUS scores mean: Adding an adjective rating scale," *Journal of Usability Studies*, vol. 4, no. 3, pp. 114-123, 2009.